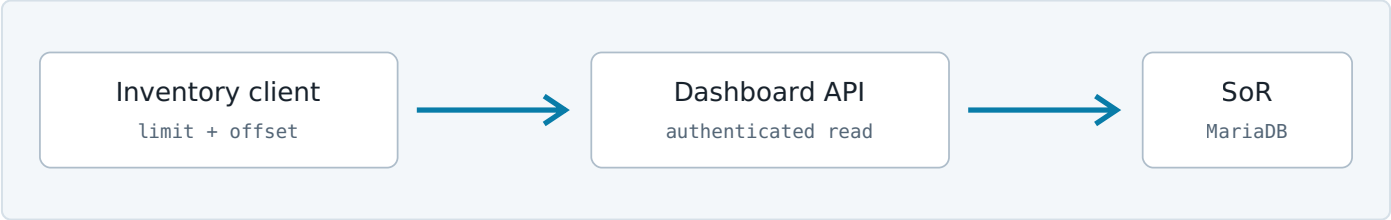
SOLARINET · SYSTEM OF RECORD

Inventory API Reference

HTTP contract for reading the inventory System of Record through the authenticated SolariNet dashboard API. This guide covers list pagination, filters, response metadata, and compatibility behavior.

Revision 1.1 · 2026-10-21 · Source: integration candidate `integrate/stability-20261020` · Internal



1. Authentication and transport

Use the dashboard's normal same-origin HTTPS session and its configured API base. Reads require an authenticated dashboard session under the deployed authorization policy. A successful response uses the common envelope `{"ok":true,"data":…,"ts":"…"}`; failures use `{"ok":false,"error":{"code":"…","message":"…"}}`. Do not send database credentials to a browser client.

2. Paginated list endpoints

GET path	Filters	List contents
`/api/inventory/components`	`stock_status`, `location_id`, `model_id`, `q`	Hardware units joined to their model.
`/api/inventory/models`	`hw_type`, `q`	Hardware model catalog.
`/api/inventory/receipts`	`vendor`, `q`	Purchase receipt metadata.
`/api/inventory/projects`	`status`, `q`	Live project records.
`/api/inventory/allocations`	`project_id`, `allocation`	Live allocations belonging to live projects.
`/api/inventory/locations?flat=1`	Pagination parameters	Flat location rows with parent IDs; use the ordinary path for the nested tree.

Add `limit` and/or `offset` to request a page. The page size must be 1–5000; the offset must be 0–2147483647. If only one parameter is supplied, the other defaults to `limit=500` or `offset=0`. Results have a deterministic order within each list.

```
GET /api/inventory/models?hw_type=server&limit=100&offset=200
```

A paginated response places rows in `data.items` and returns:

```
{
  "ok": true,
  "data": {
    "items": [{"id": 123, "make": "Example", "model": "Unit"}],
    "page": {
      "limit": 100,
      "offset": 200,
      "total": 237,
      "hasMore": false,
      "nextOffset": null
    }
  },
  "ts": "2026-10-21T00:00:00Z"
}
```

Continue while `page.hasMore` is true, using `page.nextOffset`. The total is the number of rows matching the same filters, before pagination. Inserts or deletes between requests can shift offset pages; clients that need a consistent export should use a stable snapshot workflow rather than assuming offset pages form a transaction.

3. Compatibility behavior

> **Important:** Omitting both pagination parameters preserves the pre-existing response shape: `data` is an array. Components retain their existing default limit of 500; the other list routes remain unbounded in this compatibility mode.

The combined `GET /api/inventory` endpoint retains its existing object shape—`models`, `units`, `locations`, `receipts`, `projects`, and `allocations`—for inventories with at most 500 rows in each collection. It executes bounded queries and returns HTTP 413 with `inventory_page_required` if any collection exceeds that threshold. It does not accept page parameters. Integrations that need larger inventories should call the individual paginated list paths above.

The dashboard browser adapter now requests those collection paths at 1000 rows per page, validates page totals and boundaries, and assembles the complete inventory snapshot for existing cross-reference workflows. The Components table filters and sorts the full loaded collection but renders 100 rows per page; Receipts and Projects also render at most 100 entries per page. The location hierarchy opens top-level children and lets operators expand deeper branches on demand. The adapter keeps a five-second in-memory snapshot to avoid repeating the fan-out during bootstrap and screen mount. A collection larger than 100,000 rows, or a changed total/duplicate page boundary during the read, is surfaced as an inventory load error rather than silently truncated. Same-count field updates do not have transaction snapshot isolation; reload for a fresh view.

> **Migration advice:** Request `?limit=500&offset=0` explicitly, then consume `items` and page metadata. This makes the bounded response shape explicit and leaves legacy array consumers unchanged.

4. Validation and errors

Condition	HTTP	Meaning
Malformed, zero, or out-of-range page size	400	`bad_request`; accepted page sizes are 1 through 5000.
Malformed or out-of-range offset	400	`bad_request`; accepted offsets are 0 through 2147483647.
Invalid allocation enum or project identifier	400	`bad_request`; supply a supported allocation state and positive integer ID.
Missing/expired session or denied route	401/403	Authenticate through the dashboard and observe the deployment’s role policy.

All list filters and pagination values are handled by the server. Do not interpolate user-provided values into SQL or treat a returned row as authorization to mutate it; write operations have separate authorization and CSRF checks.

5. Verification scope

The candidate’s HTTP integration test exercises list limits, page metadata, non-overlapping page boundaries, flat location pages, empty allocation pages, invalid zero limits, and the legacy aggregate’s 413 boundary against a disposable MariaDB instance. The browser adapter test traverses a two-page collection, checks stale-data behavior, and rejects over-limit and changing-total fixtures. Changed totals or duplicate IDs across pages fail the load and require a retry; page requests are not a database transaction, so same-count field updates may not be detected.